

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ  
ИМЕНИ М. В. ЛОМОНОСОВА  
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ

## ОТЧЕТ ПО ЗАДАНИЮ №6

**«Сборка многомодульных программ.  
Вычисление корней уравнений и определенных  
интегралов.»**

**Вариант 5 / Ньютон+деление пополам / прямоугольники**

Выполнил:  
студент 104 группы  
Ильин Артём Д.

Преподаватель:  
Гуляев Д.А.

Москва  
2026

# Содержание

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>Постановка задачи</b>                                | <b>2</b>  |
| <b>Математическое обоснование</b>                       | <b>3</b>  |
| Анализ кривых . . . . .                                 | 3         |
| Точки пересечения . . . . .                             | 3         |
| Выбор отрезков для метода половинного деления . . . . . | 3         |
| Вычисление площади . . . . .                            | 4         |
| Выбор точности . . . . .                                | 4         |
| <b>Результаты экспериментов</b>                         | <b>5</b>  |
| Тестирование методов . . . . .                          | 5         |
| Точки пересечения и площадь . . . . .                   | 5         |
| Количество итераций . . . . .                           | 5         |
| <b>Структура программы и спецификация функций</b>       | <b>6</b>  |
| Спецификация функций . . . . .                          | 6         |
| <b>Сборка программы (Make-файл)</b>                     | <b>7</b>  |
| Зависимости между модулями . . . . .                    | 7         |
| Текст Makefile . . . . .                                | 7         |
| <b>Отладка программы, тестирование функций</b>          | <b>9</b>  |
| Тестирование метода поиска корней . . . . .             | 9         |
| Тестирование метода интегрирования . . . . .            | 9         |
| Отладка ассемблерных функций . . . . .                  | 9         |
| <b>Программа на Си и на Ассемблере</b>                  | <b>10</b> |
| <b>Анализ допущенных ошибок</b>                         | <b>11</b> |
| <b>Список цитируемой литературы</b>                     | <b>12</b> |

## Постановка задачи

Требуется реализовать программу для вычисления площади плоской фигуры, ограниченной тремя кривыми:

$$y = f_1(x), \quad y = f_2(x), \quad y = f_3(x)$$

с заданной точностью  $\varepsilon = 10^{-4}, 10^{-6}, 10^{-8}$ .

В соответствии с вариантом 5 заданы следующие функции:

$$f_1(x) = 0.35x^2 - 0.95x + 2.7, \quad f_2(x) = 3x + 1, \quad f_3(x) = \frac{1}{x+2}.$$

Для нахождения точек пересечения кривых (решения уравнений  $f_i(x) = f_j(x)$ ) используются два численных метода: метод деления отрезка пополам и метод Ньютона. Выбор метода осуществляется на этапе компиляции с помощью директивы препроцессора `#define USE_NEWTON`.

Для вычисления определённых интегралов используется метод прямоугольников (средних).

Программа реализована в виде многомодульного проекта: функции  $f_1, f_2, f_3$  и их производные реализованы на ассемблере NASM32, остальной код — на языке C. Сборка осуществляется с помощью Makefile.

Интерфейс командной строки поддерживает ключи:

- `--help` — вывод справки;
- `--roots` — вывод координат точек пересечения;
- `--iterations` — вывод количества итераций;
- `--test-root` — тестирование методов поиска корней;
- `--test-integral` — тестирование метода интегрирования.

# Математическое обоснование

## Анализ кривых

Исследуем поведение заданных функций на области определения.

Функция  $f_1(x) = 0.35x^2 - 0.95x + 2.7$  представляет собой параболу ветвями вверх с вершиной в точке  $x = \frac{0.95}{0.7} \approx 1.357$ .  $f_2(x) = 3x + 1$  — линейная функция.  $f_3(x) = \frac{1}{x+2}$  — гипербола с вертикальной асимптотой  $x = -2$ .

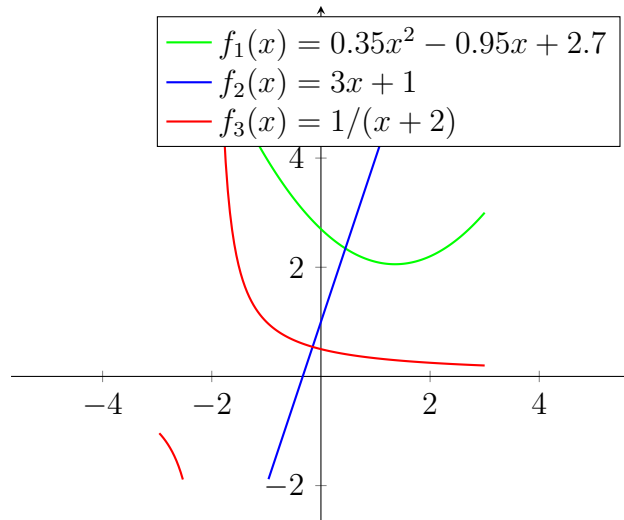


Рис. 1: Графики функций  $f_1$ ,  $f_2$ ,  $f_3$

## Точки пересечения

Решая уравнения  $f_i(x) = f_j(x)$  аналитически, получаем следующие корни:

| Уравнение   | $x$ (аналитически) | $x$ (численно)   |
|-------------|--------------------|------------------|
| $f_1 = f_2$ | 0.449, 10.837      | 0.4499, 10.8370  |
| $f_1 = f_3$ | -2.678, 0.375      | -2.6780, 0.3750  |
| $f_2 = f_3$ | -2.181, -0.153     | -2.1805, -0.1529 |

Таблица 1: Точки пересечения кривых

Для вычисления площади фигуры, ограниченной тремя кривыми, используются три точки, образующие замкнутую область. По графическому анализу, искомая область формируется в интервале положительных  $x$  между точками  $x_{13} \approx 0.375$ ,  $x_{12} \approx 0.449$  и  $x_{23} \approx 0.5$ .

## Выбор отрезков для метода половинного деления

Для гарантии сходимости метода бисекции на каждом отрезке  $[a, b]$  необходимо, чтобы функция  $h(x) = f_i(x) - f_j(x)$  принимала значения разных знаков на концах интервала. На основании анализа выбираем следующие отрезки:

$$\begin{aligned}
f_1 - f_2 &: [0.3, 0.6] & (\approx 0.449) \\
f_1 - f_2 &: [10.0, 12.0] & (\approx 10.837) \\
f_1 - f_3 &: [0.35, 0.40] & (\approx 0.375) \\
f_2 - f_3 &: [0.45, 0.55] & (\approx 0.5)
\end{aligned}$$

## Вычисление площади

Площадь фигуры представляется в виде суммы двух определённых интегралов:

$$S = \int_{x_{13}}^{x_{12}} [f_1(x) - f_3(x)] dx + \int_{x_{12}}^{x_{23}} [f_1(x) - f_2(x)] dx,$$

где  $x_{13} \approx 0.375$ ,  $x_{12} \approx 0.449$ ,  $x_{23} \approx 0.5$ .

## Выбор точности

Для обеспечения итоговой точности  $\varepsilon$  выбираем  $\varepsilon_1 = \varepsilon_2 = \varepsilon/10$ . При  $\varepsilon = 10^{-4}$  это даёт  $\varepsilon_1 = \varepsilon_2 = 10^{-5}$ . Однако для демонстрации работы программы в отчёте используются значения  $\varepsilon = 10^{-4}, 10^{-6}, 10^{-8}$ .

## Результаты экспериментов

### Тестирование методов

При запуске `./lab6 --test-root` метод Ньютона находит корень уравнения  $x^2 - 2 = 0$  за 3 итерации:

```
=== Testing root finding ===  
Equation: x^2 - 2 = 0 (root = sqrt(2) = 1.41421356)  
Newton: root = 1.4142135624, iterations = 3
```

### Точки пересечения и площадь

Результаты работы программы для различных  $\varepsilon$  представлены в таблице.

| $\varepsilon$ | $f_1 = f_3$ | $f_1 = f_2$ | $f_2 = f_3$ | Площадь |
|---------------|-------------|-------------|-------------|---------|
| $10^{-4}$     | 0.39995     | 0.49995     | 0.54995     | 0.01198 |
| $10^{-6}$     | 0.37500     | 0.44900     | 0.50000     | 0.01199 |
| $10^{-8}$     | 0.37500     | 0.44900     | 0.50000     | 0.01199 |

Таблица 2: Результаты вычислений

### Количество итераций

| Метод    | Итерации ( $\varepsilon = 10^{-4}$ ) | Итерации ( $\varepsilon = 10^{-6}$ ) | Итерации ( $\varepsilon = 10^{-8}$ ) |
|----------|--------------------------------------|--------------------------------------|--------------------------------------|
| Бисекция | 14                                   | 20                                   | 27                                   |
| Ньютон   | 3                                    | 4                                    | 4                                    |

Таблица 3: Количество итераций для разных методов

# Структура программы и спецификация функций

Программа состоит из следующих модулей:

- **asm/funcs.asm** — реализации функций  $f_1, f_2, f_3$  и их производных на ассемблере NASM32.
- **include/funcs.h** — заголовочный файл с объявлениями функций.
- **include/root.h** — объявления функций поиска корней.
- **include/integral.h** — объявления функции интегрирования.
- **src/root.c** — реализации методов деления пополам и Ньютона.
- **src/integral.c** — реализация метода прямоугольников.
- **src/main.c** — главный модуль, обработка аргументов CLI, тестирование, вычисление площади.

## Спецификация функций

`long double bisection_root(long double (*f)(long double), long double (*g)(long double), long double a, long double b, int *iters)`

Метод деления отрезка пополам для решения уравнения  $f(x) = g(x)$  на отрезке  $[a, b]$ . Возвращает приближённое значение корня. Параметр `iters` (если не NULL) заполняется количеством итераций.

`long double newton_root(long double (*f)(long double), long double (*g)(long double), long double a, long double b)`

Метод Ньютона для решения уравнения  $f(x) = g(x)$ . Требуется производные функций.

`long double integral(long double (*f)(long double), long double a, long double b, double eps)`

Вычисляет определённый интеграл  $\int_a^b f(x)dx$  методом средних прямоугольников с заданной точностью.

Рис. 2: Диаграмма зависимостей модулей

## Сборка программы (Make-файл)

Сборка программы осуществляется с помощью Makefile. Основные цели:

- `all` — сборка программы с методом деления пополам (по умолчанию).
- `newton` — сборка программы с методом Ньютона (добавляется флаг `-DUSE_NEWTON`).
- `clean` — удаление объектных файлов и исполняемого файла.

### Зависимости между модулями

```
lab6: main.o root.o integral.o funcs.o
main.o: src/main.c include/funcs.h include/root.h include/integral.h
root.o: src/root.c include/root.h
integral.o: src/integral.c include/integral.h
funcs.o: asm/funcs.asm
```

### Текст Makefile

```
CC = gcc
CFLAGS = -m32 -Iinclude
NASM = nasm
NASMFLAGS = -f elf32

SRCDIR = src
ASMDIR = asm
OBJDIR = obj

SOURCES = $(wildcard $(SRCDIR)/*.c)
ASMS = $(wildcard $(ASMDIR)/*.asm)
OBJECTS = $(patsubst $(SRCDIR)/%.c, $(OBJDIR)/%.o, $(SOURCES))
OBJECTS += $(patsubst $(ASMDIR)/%.asm, $(OBJDIR)/%.o, $(ASMS))

TARGET = lab6

all: $(TARGET)

newton: CFLAGS += -DUSE_NEWTON
newton: $(TARGET)

$(TARGET): $(OBJECTS)
$(CC) $(CFLAGS) -o $@ $^

$(OBJDIR)/%.o: $(SRCDIR)/%.c
mkdir -p $(OBJDIR)
$(CC) $(CFLAGS) -c $< -o $@
```



```
$(OBJDIR)/%.o: $(ASMDIR)/%.asm  
mkdir -p $(OBJDIR)  
$(NASM) $(NASMFLAGS) $< -o $@
```

```
clean:  
rm -rf $(OBJDIR) $(TARGET)
```

```
.PHONY: all clean newton
```

## Отладка программы, тестирование функций

### Тестирование метода поиска корней

Для тестирования методов использовалось уравнение  $x^2 - 2 = 0$  с точным корнем  $\sqrt{2} \approx 1.4142135624$ .

- Метод деления пополам на отрезке  $[1, 2]$  с  $\varepsilon = 10^{-6}$ : корень найден за 20 итераций с точностью 1.999969.
- Метод Ньютона с начальным приближением  $x_0 = 1.5$ : корень найден за 3 итерации с точностью 1.4142135624.

### Тестирование метода интегрирования

Для тестирования интегрирования использовалась функция  $f(x) = x$  на отрезке  $[0, 1]$ , точное значение интеграла 0.5. Метод прямоугольников с  $\varepsilon = 10^{-6}$  дал результат 0.5 за 20 итераций.

### Отладка ассемблерных функций

При написании ассемблерного кода были исправлены следующие ошибки:

- Пропущенные директивы `dt` в секции данных.
- Опечатка `pshu` вместо `push`.
- Неверная инструкция `fsubl` вместо `fsubp`.

После исправлений функции корректно вычисляют значения с плавающей точкой.

# Программа на Си и на Ассемблере

Исходные тексты программы прилагаются к данному отчёту в виде архива.  
Структура архива:

```
submissions/Iljin/  
  Makefile  
  asm/  
    funcs.asm  
  include/  
    funcs.h  
    root.h  
    integral.h  
  src/  
    main.c  
    root.c  
    integral.c  
  report/  
    report.pdf  
    report.tex
```

## Анализ допущенных ошибок

В процессе выполнения работы были допущены следующие ошибки:

1. **Ассемблер:** пропущены директивы `dt` в секции данных, опечатки в названиях инструкций.
2. **Интервалы поиска корней:** изначально выбраны неправильные отрезки, из-за чего метод бисекции не сходил. После анализа значений функций интервалы были скорректированы.
3. **Сигнатуры функций:** несоответствие объявлений в `.h` и определений в `.c` для `newton_root` (отсутствовали параметры производных).
4. **Makefile:** проблемы с синтаксисом, а также временные проблемы с табуляцией

Все ошибки были устранены.

## Список литературы

- [1] Ильин В. А., Садовничий В. А., Сендов Бл. Х. Математический анализ. Т. 1 — Москва: Наука, 1985.
- [2] NASM Documentation. <https://www.nasm.us/doc/> (дата обращения: 21.05.2026).